

highlightr: Creating Highlighted Text Based on Source Frequency

by Rachel Rogers and Susan VanderPlas

Abstract Analysts sometimes need to compare a source document to documents derived from it; examples include comparing notes to an original source document, or when examining consecutive edits (such as Wikipedia article histories). By comparing multiple derivative documents to a source document, researchers can evaluate which portions of the source document are the most preserved in its derivative forms, which may indicate information regarding the importance of these phrases. The **highlightr** package provides analysis and visualization methods to map derivative documents to their source document in the form of a ‘heatmap’ of preserved information. Sequential documents are compared using collocations, or phrases of a set length. Once the frequency with which each phrase appears in derivative documents is calculated, the frequency can be mapped to a corresponding color to create a highlighting gradient. **highlightr** streamlines this process and integrates the source document and heatmap in a useful visual display.

1 Introduction

An individual’s notes can provide valuable insight into their process when learning new information. Comparing note sheets with the presented information allows researchers to determine which areas participants found worth recording. This process is particularly important when conducting studies of how jurors interpret legal arguments, using legal transcripts instead of more costly mock-trial videos. The **highlightr** package (Center for Statistics and Applications in Forensic Evidence et al., 2024) provides a set of functions developed for analyzing and visualizing participant notes to indicate which portions of the transcript were most informative.

The analysis methods in **highlightr** are useful well beyond the legal transcript study for which they were developed. The same methods can also be used to compare other documents which have a similar source, such as abstracts for conference presentations and journal papers on the same topic, or sequential edits to a Wikipedia page. In this paper, we demonstrate the use of the **highlightr** package across a number of different document comparison problems, discussing the design and function of the package as well as the adjustments which must be made as we take on problems with different source-document relationships.

By establishing similarity to the source text, we can determine which portions of the document were judged to be important and use this information to create visualizations highlighting important sections of the document based on users’ notes. This similarity is based on finding the occurrence of phrases from the source document in the derivative documents; the higher the number of occurrences, the more intense the highlighting.

2 Related work

Single document forms of text visualization include the construction of word clouds, where a cluster of frequently used words are shown, with larger words indicating more frequent occurrence. In R, word clouds can be implemented using **wordcloud** (Fellows, 2018). Similarly, frequency values can be shown directly in a barplot (Shibuya and Jensen, 2015). While word clouds and frequency barplots provide useful information about the individual frequency of words in a collection of texts, the surrounding sentence context is not taken into account. Silge visualize gendered verbs used in film scripts through digrams, or sequences of two words, where the first word consisted of a pronoun (“he” or “she”) and the second word was a screen direction. They then plotted the frequency of each word by gendered pronoun in a diverging barplot using **ggplot2** (Wickham, 2016). This text visualization has added context in the form of pronouns, when compared to word clouds. It provides a useful visualization of gender differences present in film scripts. In a more general sense, bigrams can also be visualized (and analyzed) in a graphical network, where segments indicate that two words form a bigram (Shibuya and Jensen, 2015). When evaluating a text as a whole, Shibuya and Jensen (2015) also use dispersion plots to depict the locations of n-grams, or phrases of length n, throughout a text. In a dispersion plot, one axis corresponds to the word count, and a line is drawn at the point that the n-gram appears, resulting in a barcode-like appearance that indicates the locations of the specified n-gram.

The goal of **highlightr** is not to summarize a single document, but to visualize the relationship

between a source document and its derivatives. This comparison of source and derivative documents is similar to the work of Ben Fry, who compared various editions of Charles Darwin's *On the Origin of Species* (Fry, 2018). In this work, Fry (2018) uses color to denote additions and animates deletions between six editions of Darwin's work, and notes how the language and structure change over time.

Similarly, Jänicke et al. (2014) utilize visualization tools in JavaScript to compare versions of the Bible for text reuse at a sentence level. Pairwise document comparisons are accomplished through a grid format, where a document is presented along each axis in sequential sections and similarities are shown in a heatmap form. They also provide an alternative visualization at the sentence-level in Dot Plot form, where a dot's location on the graph indicates a sentence pair. Alternatively, when viewing two texts side by side, similarities are connected in a parallel coordinate plot format, where a line is drawn between similar sentences. Jänicke et al. (2014) also perform sentence-level comparisons between multiple texts using sentence alignment flow, where a base sentence is depicted with variations connected via lines.

Comparison between multiple documents also bears some resemblance to plagiarism detection, where portions of texts are compared with references to determine if the text was copied. Parker and Hamblen (Parker and Hamblen, 1989) describe an algorithm that uses the character differences to compare similarities in computer programs, as well as other code-specific features.

Piao and McEnerly (2003) describes the TESAS algorithm, used for detecting relationships between two documents at the sentence level, with a focus on applications in journalism. This algorithm relied on shared strings, stems and synonyms for evaluating similarity between sentences in order to produce a score. The accompanying tool provided side-by-side comparisons of source and derivative sentences. Similarly, Olsen et al. (2011) describe the application of sequence alignment algorithms, traditionally used in bioinformatics, to compare passages of text. To visualize the correspondences between texts, Olsen et al. (2011) place texts side-by-side with color-coded similarities.

Visualizations for source and derivative text comparisons usually consist of texts placed next to each other, with color to indicate similarities and differences (Olsen et al., 2011; Fry, 2018; Piao and McEnerly, 2003). This type of data display uses the method of juxtaposition, which requires individuals to remember the content of the first document when comparing to the second document (Gleicher et al., 2011). These displays are limited in the number of texts that can be compared, as well as the length of texts, given the mental resources necessary for more complex comparisons. In contrast, the **highlightr** method focuses on a single "source" text, where similarities to derivative documents are directly mapped. This method relies on explicit encoding (Gleicher et al., 2011), since the relationship of interest is directly mapped in the visualization. While this reduces the amount of information displayed, in that all derivative documents are not presented alongside the source document, the method provides a simpler display of similarity.

This display method borrows from the methods seen in single-document text analysis. Similar to wordclouds and barplots, the output of interest is frequency. Instead of considering the frequency of words, the method considers the frequency of n-grams and the location of these n-grams throughout the document, much like dispersion plots (Shibuya and Jensen, 2015). However, this visualization differs in that its function is to visualize an entire document, and the frequency of this source document's n-grams in derivative documents, in order to provide a cohesive picture of the most borrowed phrases.

The collocation approach used in **highlightr** is similar to the shingling approach, where overlapping sequences of words (or 'shingles') are compared across documents (Olsen et al., 2011; Smith et al., 2013). Unlike longest common substring approaches (Amir et al., 2018; Crochemore et al., 2017), where the longest common string of characters between two texts is identified, the shingle (and collocation) approach focuses on finding matches in a source text of phrases of a set length. This allows for source text visualizations to be created based on the frequency for all phrases (or collocations) in a document, providing a view of the document as a whole.

While the word "collocation" traditionally refers to words that occur more frequently together - such as "white house" (Merriam-Webster, 2023), tools for collocation analysis helpfully provide the frequency of occurrence for strings of a specified length. This method allows us to calculate the frequency of word sequences (or n-grams) in derivative documents, and map these word frequencies back to the corresponding word sequence in the source document. When the word sequences of the derivative documents do not directly correspond to the source document, fuzzy matching can be used to find the closest matching source document sequence. This approximate string matching allows for the inclusion of inexact strings (Green, 2025).

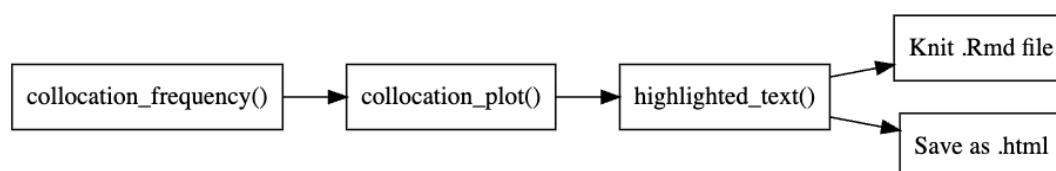


Figure 1: A flowchart of highlightr functions used to generate highlighted html output. Arrows indicate inputs.

3 Highlightr

3.1 Collocations

Collocation analysis calculates the frequency of sequentially-appearing word strings (n-grams) to identify words that tend to ‘stick together’. This process has been used to assist in identifying common multi-word phrases with a set meaning, such as “ice cream”. In this analysis, the collocation function of [quanteda](#) ([Benoit et al., 2018](#)) is used to consider longer strings ([Schweinberger, 2023](#)). Each set of words, or collocation, is found for the source document, then located in the derivative documents to calculate a frequency for each phrase.

Once sequences of words from derivative documents have been mapped to the source document, the average frequency is taken per word. When a collocation appears multiple times in the same source document, the derivative document frequency is divided by the number of appearances in the source document. To create a continuous highlighting effect (without breaks between words), each word is weighted by the collocations containing that word. Initially, each word is assigned a single color using **ggplot2** color scale mappings from frequency to hue. Then, HTML gradient boxes are created for each word, where the left color corresponds to the frequency of the previous word, while the right color corresponds to the frequency of the current word, producing a smooth visual transition between each word.

The process for creating highlighted text is depicted in Figure 1. The `collocation_frequency()` function performs the following operations: tokenizes the derivative and source documents; summarizes collocation frequencies in reference to the tokenized source document; maps collocation frequencies to the fully formatted source document and averages the collocation count for each word. Collocation frequencies are calculated by either using only direct matches or including indirect matches with `fuzzy=TRUE`. This function provides the averaged frequency values for each word, which lends itself to further analysis and visualization methods. For example, frequency values can be graphed in the order that they appear in the text to provide an overview of the document, as seen in Figures 4 and 5, which compare the effect of tuning parameter values. When used interactively, this visualization method can make it easy to detect phrases with extreme frequency. However, the entirety of the text is not easily included in the visualization. For this purpose, the frequencies must be transformed into a gradient and mapped onto the source document.

The `collocation_frequency()` object is then used by `collocation_plot()` to create a plot object with corresponding highlight colors. Other objects that include a column for the desired text, word order, and value of interest can also be inputted into `collocation_plot()` to generate color values for each word. The final step is to input this object into `highlighted_text()` to appropriately format the html tags to create the highlighting effect. This output can then be viewed through .Rmd by including the object outside of a code chunk, or through outputting the object as an .html file.

The visualization generated from this output follows common visualization design principles, such as strategic use of color, simplicity in design, and careful consideration of *what* the visualization is meant to convey ([Midway, 2020](#)). In contrast to using a table to represent frequencies, this visualization allows for the comparison of values ([Gelman et al., 2002](#)) within a source document, which provides the audience with a whole-of-document perspective of the data. This focus on the source document’s entirety with reference to several derivative documents simultaneously directly informs the visualization method implemented. Additionally, the use of semantically-resonant colors has been shown to increase performance in graphical evaluations ([Lin et al., 2013](#)). Therefore, the color yellow is used to represent higher frequency counts, as it is associated with traditional highlighters.

3.2 Fuzzy matching

The [zoomerjoin](#) package ([Green, 2025](#)) facilitates fuzzy matching on the indirect collocations - participant edits such as spelling errors, abbreviated words, or otherwise minor changes within a sentence. Leveraging fuzzy matching allows these minor changes to be matched back to their source using

Table 1: Similarities and differences between ‘rabbit’ and ‘rabid’ using n-grams of length 3

Relationship	rab	abb	bbi	bit	abi	bid
Intersection	1	0	0	0	0	0
Union	1	1	1	1	1	1

Jaccard similarity as approximated through the MinHash algorithm (Green, 2025). Two parameters, `n_bands` and `band_width`, are used in **zoomerjoin** to control the number of bands used and the width of the bands in the MinHash algorithm when computing fuzzy matches. These parameters can be passed to **zoomerjoin** through specifications in `collocation_frequency()` in order to adjust the comparison coverage of the algorithm.

Jaccard similarity is the intersection of two sets divided by the union of the sets (Wu et al., 2022). In this application, each collocation is considered a set. The units of interest are specified as n-grams in **zoomerjoin**. Green (2025) recommend to set a size of 2 or 3 for names, and a size of 5 or 6 for sentences. Because collocation length is usually longer than a name but shorter than a sentence, the default for fuzzy matching in **highlightr** has been set at 4 characters. This means that a subset of 4 characters in the two strings being compared will be counted in the intersection if they are identical.

For example, take the two following strings: “rabbit” and “rabid”. With a three-character n-gram width, the similarity between the two words can be visualized as Table 1. Here, there is a single common 3-gram (‘rab’), and a total of six 3-grams between the two words. This would result in a Jaccard distance of $\frac{1}{6} = 0.167$, indicating that the similarity between the two words is low.

Because the intent of the derivative document is clearer with simple typos (such as a single letter change) than with larger differences (such as changing multiple words), the count for these fuzzily matched observations are weighted by the Jaccard similarity value. A Jaccard similarity value of 1 would indicate identical strings, while a Jaccard similarity value of 0 would indicate no similarity between strings. The default threshold to be considered a match in **highlightr** is 0.7, corresponding to the default threshold used in **zoomerjoin** (Green, 2025).

Another factor in the calculation is the number of closest matches per ‘misspelled’ collocation. Some derivative collocations may have the same Jaccard similarity value for multiple source document collocations. In these cases, the frequency of the ‘misspelled’ derivative collocations are split evenly between the closest source document collocations, resulting in a total fuzzy frequency per source document collocation of:

$$f_i(x) = \sum_{i=1}^n \frac{x_i * d_i}{n_i} \quad (1)$$

where x_i is the count for derivative document collocation i , d_i is the Jaccard similarity value between the source document collocation and the ‘misspelled’ derivative document collocation i , and n_i is the number of closest source document collocation matches for derivative document collocation i . This quantity is then added to the total of direct collocation matches, before dividing by the number of times the collocation appears in the source document.

The Jaccard method is implemented due to simplicity and interpretability. This method is meant to capture misspellings, and provides a built-in method of weighting indirect matches in a way that is easy to explain and add to direct match frequency values. Other methods for detecting semantic similarity, such as the **word2vec** package (Wijffels and Watanabe, 2025) or Bidirectional Encoder Representations from Transformers (BERT) (Devlin et al., 2019), may be considered in conjunction with this method. However, the interpretability of the output would be less clear-cut than the 0-1 similarity value output of the Jaccard method.

3.3 Optimizing collocation length

The default collocation length in **highlightr** is 5. This length was chosen as a default to balance two considerations: the frequency with which a phrase may appear in the source document, and the number of sequential words that may appear in derivative documents. If the phrase is too short, a phrase may appear multiple times throughout the source document, which would complicate mapping the phrase back to a specific portion of source document. For example, setting the collocation length to 2 would pick up phrases such as “in the”, which may occur multiple times in a single document. On the other hand, it is unlikely that a large amount of derivative documents would have an exact 10-word phrase, especially if the documents represent notes or abbreviations. In a newspaper article matching scenario, Smith et al. (2013) utilize a collocation length between 5 and 7 for their shingling approach, and discuss the tradeoff between allowing flexibility in textual variation through a shorter

length and detection of fixed phrases.

The appropriate collocation length may differ based on the length of the source document, the number of derivative documents, and the type of derivation; notes, subsequent versions, and other types of derivatives may have different expected collocation lengths.

We use the following equation in order to measure the smoothness of transitions between words:

$$S(f) = \frac{\sum_{i=1}^{m-1} (w_i - w_{i+1})^2}{(m-1) * k} \quad (2)$$

Here, w_i is the average collocation frequency for word i of the source document collocation (where frequency is computed either using the fuzzy matching shown in Equation (1), or using non-fuzzy matches), m is the number of words in the source document, and k is the number of derivative documents. By computing the sum of squared distance between the frequencies of consecutive words, we can evaluate the smoothness of transitions between words in the document on the whole. Dividing by $m-1$ and k provides a form of standardization, as we would expect a larger sum of squares when there are more words and when the frequency values are higher (e.g. there are more derivative documents).

If the collocation length is too long, the smoothness parameter will approach 0, meaning that the entire document will be the same color, which limits the utility of visualizing collocations. However, when collocation length is short, there are large differences in the collocation counts of sequential words, indicating that whole phrases are not being recognized.

Equation (2) produces an L-shaped curve resembling exponential decay, where the shortest collocation graphed (length 2) has a high value that quickly decreases. The goal is to find a collocation of minimum length that results in ‘smooth transitions’; functionally, as with many parameter optimization problems, this means identifying an ‘elbow’ in the curve where both collocation length and variation between collocation counts are acceptably low. This ‘elbow’ or ‘knee’ approach is often used in k-means clustering (Schubert, 2023) and operations research (Thomas and Sheldon, 1999). It can be defined as the location where the slope of the curve changes from large to small (Thomas and Sheldon, 1999). While the best method for identifying the elbow of a curve is contested (Schubert, 2023), visualization provides a “useful clue” (Thomas and Sheldon, 1999) for collocation length values that can be more thoroughly explored.

4 Practical applications of highlightr

4.1 Abstract comparison

In the first example, we compare the abstract from the author’s dissertation (Rogers, 2024) (source document) to conference presentations given on portions of the same material (derivative documents). Since the presentation abstracts cover portions of the same content as the dissertation, we would expect to see some overlap between the subjects and wording. This comparison provides a concise example of how **highlightr** functions, since abstracts are generally limited to less than 200 words. In this case, since we do not expect an exact match between the wording in different summaries, we use fuzzy matching to include indirect matches.

The `collocation_frequency()` function compares the source and derivative documents. It is assumed that both the source and derivative texts are located in a single data frame. The row corresponding to the source document, as well as the column containing the text to be compared, must be specified in the function. The function first tokenizes the source and derivative documents, a process of splitting the text in the document into words, allowing for a comparison across documents. Next, the tokenized source and derivative documents are compared using collocations.

```
# Read in data
abstract <- readr::read_csv("data/abstracts.csv")

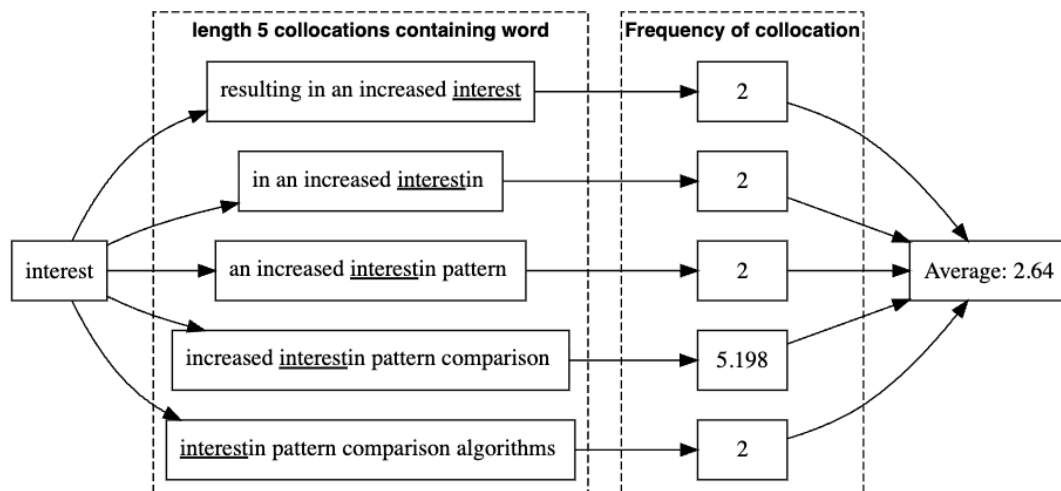
# Calculate collocation frequency using fuzzy matching and map to source document

collocation_abstract <-
  collocation_frequency(abstract,
    source_row=which(abstract$Conference=="Dissertation"),
    text_column = "page_notes", fuzzy=TRUE)
```

Table 2 shows the fuzzy collocation results for a collocation of length 5 (the default collocation length). These results are organized by the first word of the collocation. The “Word Location in

Table 2: Dissertation Collocations of Length 5

Word	Word Location in Collocation					First Collocation	Frequency
	1st	2nd	3rd	4th	5th		
Subjective	2	NA	NA	NA	NA	subjective pattern comparison has been	2
pattern	2	2	NA	NA	NA	pattern comparison has been subject	2
comparison	2	2	2	NA	NA	comparison has been subject to	2
has	2	2	2	2	NA	has been subject to increased	2
been	2	2	2	2	2	been subject to increased scrutiny	2
subject	2	2	2	2	2	subject to increased scrutiny by	2

**Figure 2:** A flowchart of the collocation process for the word ‘interest’ with collocation length 5.

Collocation” identifies the word’s place in the collocation frequency calculated. The “1st” column provides the collocation frequency for the collocation that begins with the word in the “Word” column (shown as the “First Collocation”). Note that the first word of the source document (“Subjective”) only appears in a single collocation, resulting in empty values for the remaining “Word Location in Collocation” columns. Similarly, the second word of the source document (“pattern”) appears in two collocations, resulting in two collocation values. The first frequency value corresponds to the collocation where “pattern” is the first word: “pattern comparison has been subject”, while the second frequency value corresponds to the collocation where “pattern” is the second word: “Subjective pattern comparison has been”. This pattern continues for the rest of words in the source document. The “Frequency” column is the average of the collocations that a word appears in - this is the value used when assigning color to the text.

This collocation process is outlined in the flowchart in Figure 2, corresponding to the word ‘interest’ in the first sentence of the source document abstract: “Subjective pattern comparison has been subject to increased scrutiny by the courts and by the general public, resulting in an increased **interest** in pattern comparison algorithms that provide quantitative assessments of similarity for use by forensic scientists.” Because the collocation is of length 5 and the word is not at the beginning of the document, there are five collocations that contain this instance of the word ‘interest’. While most of these collocations have a frequency of 2, meaning that they appeared in approximately two derivative documents (while accounting for indirect matches), we can see that the phrase “increased interest in pattern comparison” has a higher frequency count, when fuzzy matches are included, for a value of approximately 5.198. This higher frequency count would affect all words contained in the collocation, creating a more yellow highlight around the phrase. The values for collocations containing the word ‘interest’ are averaged together to provide the number used in the highlighting effect: 2.64.

This data is then assigned a gradient corresponding to the frequency through **ggplot2** using `collocation_plot()`, and the gradient is converted into html tags using `highlighted_text()`:

```
freq_plot <- collocation_plot(collocation_abstract)
page_highlight <- highlighted_text(freq_plot)
```

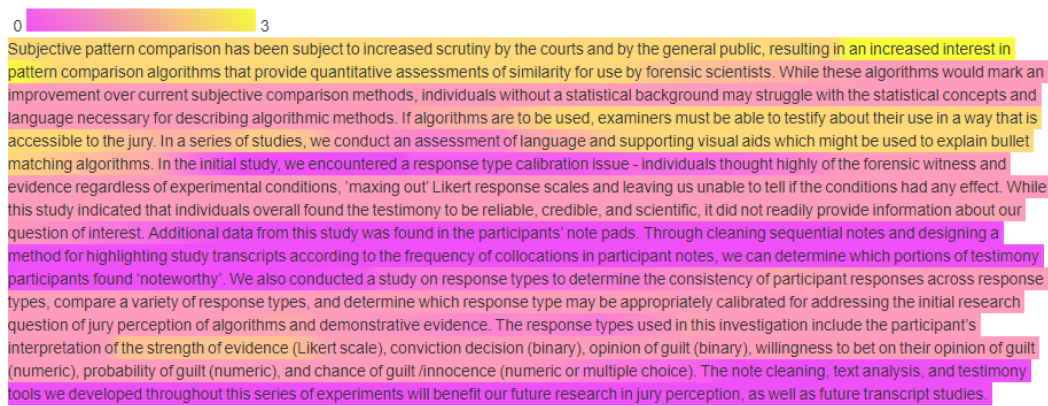


Figure 3: Source Document with Collocation Length 5 using Fuzzy Matching and Default n-gram Width of 4

This results in the output shown in Figure 3. Here, the highest weighted frequency is around 3, with particular emphasis on the phrase “an increased interest in”. Two tuning adjustments that can be made in the case of fuzzy matching are a change in n-gram width (specified through `n_gram_width`) and a change in the threshold of what is considered a match (specified through `threshold`). The default parameter for n-gram width is 4, meaning that the fuzzy matching process considers sequences of 4 characters when computing Jaccard similarity. The default parameter for threshold is 0.7, meaning that a Jaccard value of 0.7 or higher is necessary to be considered a ‘match’.

A comparison of frequency values for thresholds ranging from 0.5 to 0.95 and n-gram widths ranging from 2 to 10 are shown in Figures 4 and 5, respectively. Frequency values for the default settings are shown in red. In both cases, frequency values are similar across tuning parameters, with most settings following similar trends. There is some divergence in both cases around the phrase “algorithms and demonstrative evidence” (as shown in Figures 4 and 5), where lower set values demonstrate slightly higher frequencies. There is also a high peak around the phrase “bullet matching algorithms” for smaller set values, indicating that relaxing ‘match’ qualifications results in higher frequency values for this phrase. While the overall trend is similar, different values chosen for these tuning parameters can result in different emphasis for specific phrases. These options of n-gram width and threshold are only available in the case of fuzzy matching, as it determines the computation of the Jaccard similarity.

Alternatively, non-fuzzy matching can also be used by excluding the `fuzzy=TRUE` argument from `collocation_frequency()`, as follows:

```
collocation_abstract <-
  collocation_frequency(abstract,
                        source_row=which(abstract$Conference=="Dissertation"),
                        text_column = "page_notes")
```

This provides a document that only includes direct matches between collocations, shown in Figure 6. When indirect matches are excluded, the maximum value on the scale decreases from four to two, and there is an equal highlighting throughout most of the initial sentence. While the removal of fuzzy matches results in the exclusion of close phrases, the nonfuzzy document has a clearer interpretation. The count directly represents the average number of times the collocations associated with a word appeared in derivative documents, whereas fuzzy match counts are the average of a weighted collocation frequency.

We compare the collocation smoothness curve to the text itself in order to find a collocation length that is acceptably smooth. Figure 7 shows the relationship between collocation length and smoothness, where 5 is marked as the default collocation length. As this graph demonstrates, smoothness values quickly decrease as the collocation length increases, with values approaching 0. In this case, a collocation of length 5 does not appear to be ill-suited to the graph. One limitation of this example is the small sample size - we only have 7 derivative documents. This results in the small smoothness values on the y-axis, as shown in Figure 7.

Figure 8 shows the dissertation abstract when the collocation length is 3. In this case, it is difficult to pick out main passages from the document - “increased interest”, “matching algorithms”, and “demonstrative evidence” appear to be commonly repeated, but this provides a limited view of the document as a whole. While the maximum count here is 4, this information seems like it could have been better represented as a table of collocations and frequencies.

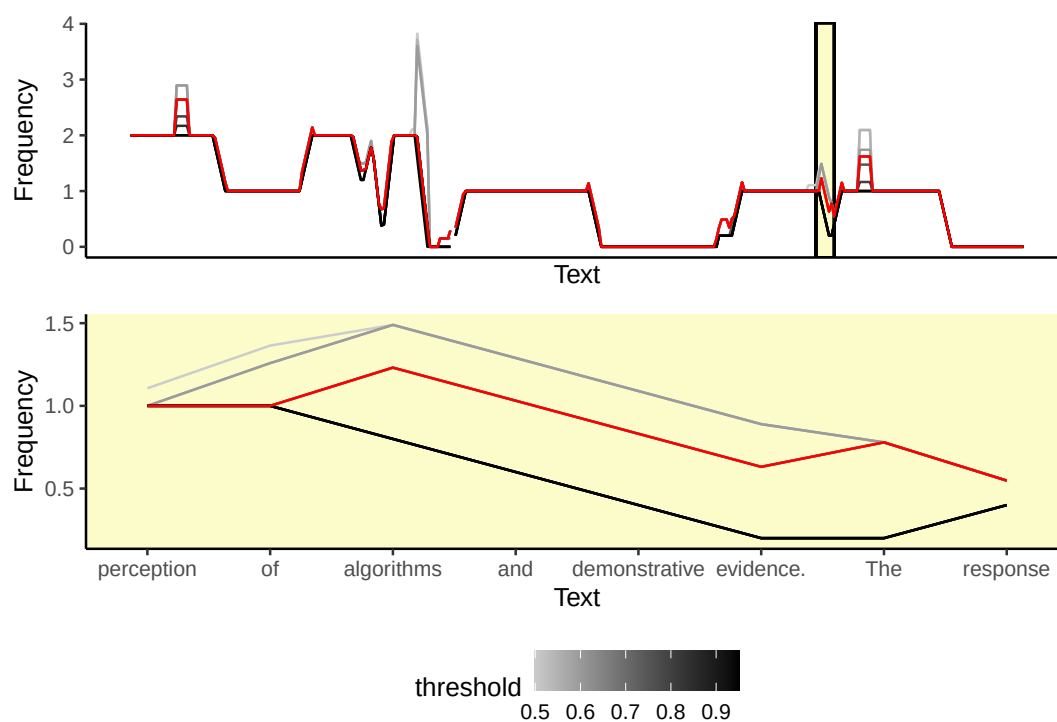


Figure 4: Graph of Frequency Values by Word for Thresholds from 0.5 to 0.95 with Collocation Length 5 using Fuzzy Matching.

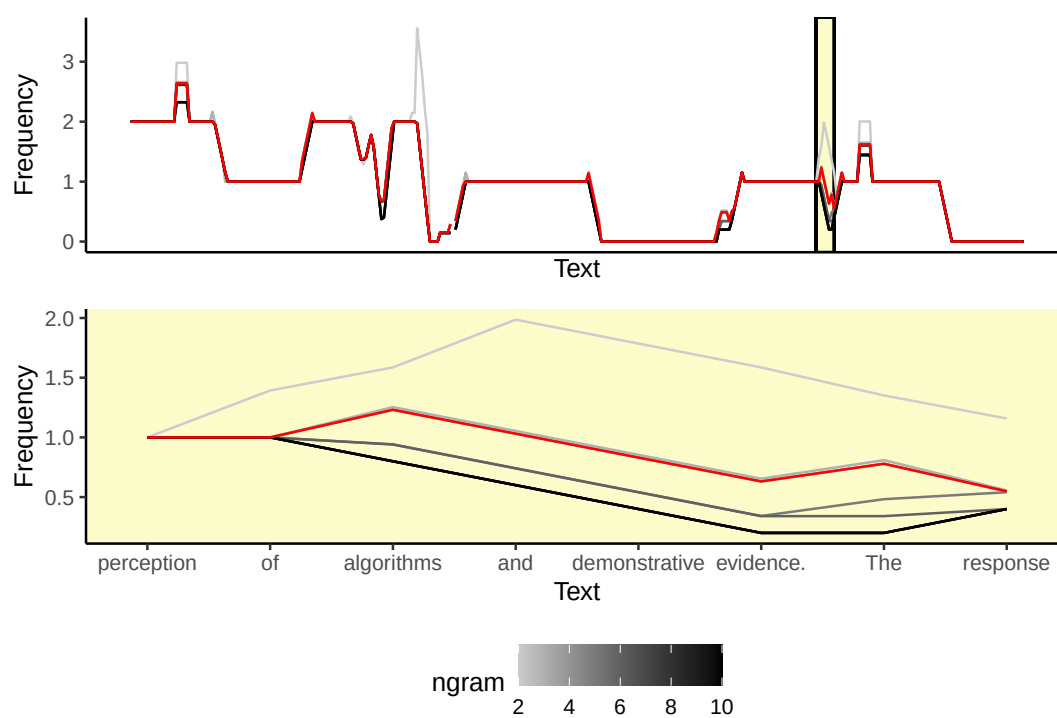


Figure 5: Graph of Frequency Values by Word for N-Gram Widths from 2 to 10 with Collocation Length 5 using Fuzzy Matching.

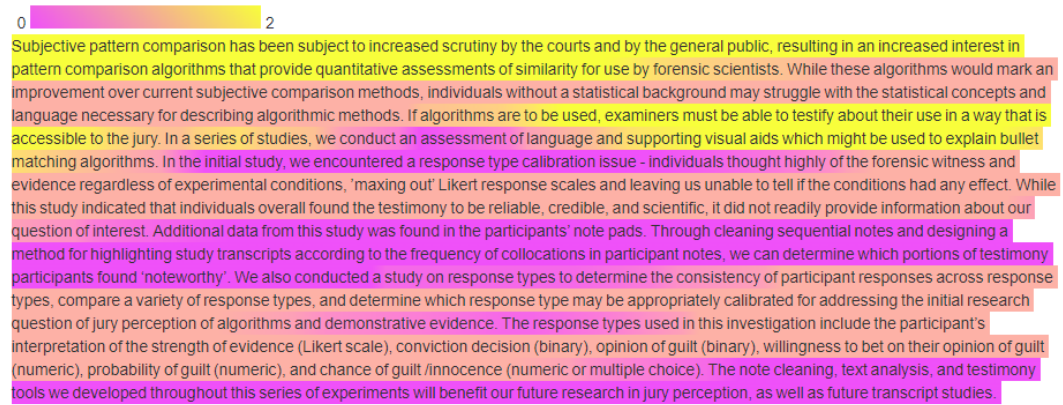


Figure 6: Source Document with Collocation Length 5 without Fuzzy Matching

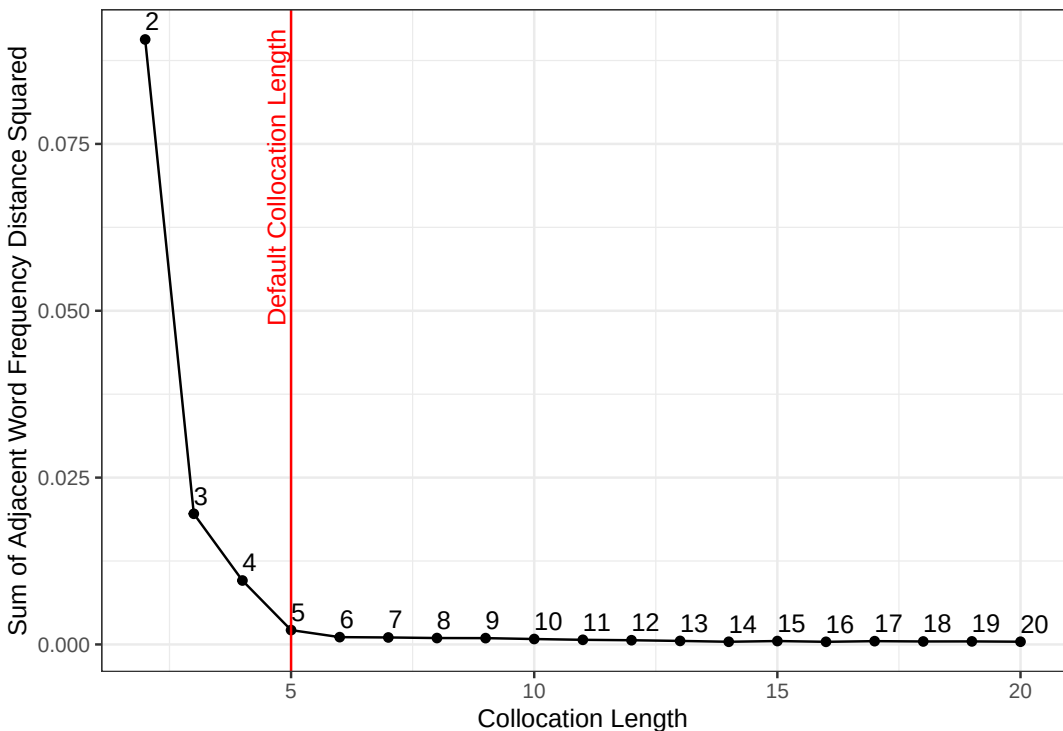


Figure 7: Smoothness of Summary Highlighting in Abstract Example

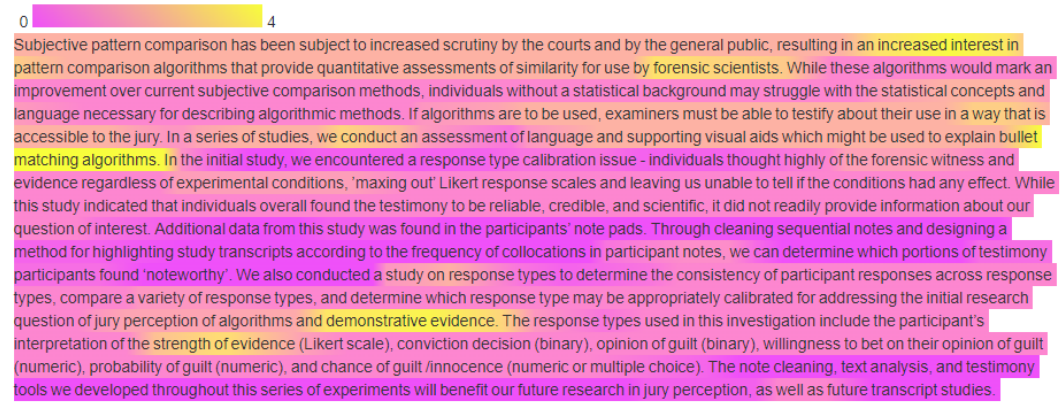


Figure 8: Source Document with Collocation Length 3 using Fuzzy Matching

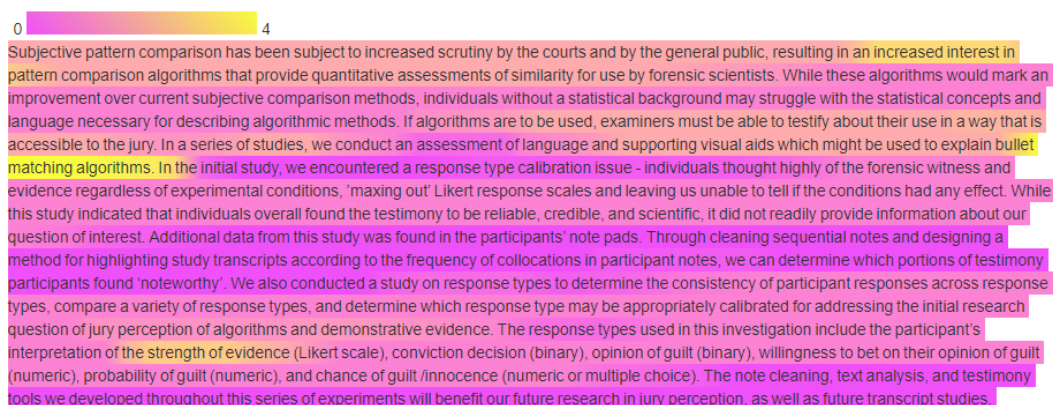


Figure 9: Source Document with Collocation Length 4 using Fuzzy Matching

Increasing the collocation length to 4 results in Figure 9. This graphic resembles Figure 8, but features less emphasis on short phrases such as “demonstrative evidence” and “forensic scientists”. The emphasis on “bullet matching algorithms”, however, is still strong.

By further increasing the collocation length to 5, shown in Figure 3, there are smoother transitions and more focus on phrases than was seen in Figure 9. Smoother transitions are particularly evident in the first sentence of the document, which is similar across several documents. We can begin to see overall themes of the work which were highlighted in different conference presentations - pattern comparison, jury accessibility, and strength of evidence. Phrases that were not replicated in derivative documents are also clearer: ‘response type calibration’, ‘cleaning sequential notes’, and ‘future research in jury perception’.

Based on these images, a collocation of length 5 balances the smoothness needed for fluid interpretation of this data and the short length necessary to capture as many observations as possible. This length is also located in the elbow of the distribution. While a collocation of length 5 is shown as a good fit for this application of the function, other applications may benefit from the selection of a different collocation length. The optimization method described above can be applied to explore collocation values in other applications, and tuning plot comparisons such as Figures 4 and 5 can be created to visualize differences in parameter values.

4.2 Wikipedia edits: Tim Walz

highlightr can also be applied to analyze Wikipedia edit history. Often, political candidates’ pages will be cleaned up as they announce their entry into a race; this edit frequency can sometimes be informative when US presidential candidates announce their selection of a vice president. Here, we demonstrate the use of **highlightr** to examine former Vice Presidential Candidate Tim Walz’s Wikipedia page (Tim, 2024), comparing Wikipedia edits from approximately August 15, 2024 to August 29, 2024 to edits from before he was nominated as a vice presidential candidate. We scraped the 150 most recent versions of Tim Walz’s Wikipedia article on August 29, 2024 (resulting in articles dating back to August 18, 2024), as well as 150 articles from before he was nominated (with a start date of July 31, 2024, and an end date of April 22, 2023). Due to the large amount of edits around Walz’s nomination, these edits were not included in this analysis.

In this analysis, we aim to determine how his article was “cleaned up” for the presidential election, and whether information was added or removed to improve his public profile. Figure 10 shows the frequency of edits to Walz’s article by date. There are many more edits in the later part of 2024 than in earlier time periods. This is especially noticeable after Walz was nominated.

The most recent article and the oldest article in record are both used as source documents for this comparison. The highlighted versions of these documents are shown in Figure 11. As this figure demonstrates, the length of Walz’s Wikipedia article has doubled. Through the use of **highlightr**, we can see that this increase in length is due not only to recent news in the 18-month period between the newest and the oldest article, but also to the inclusion of older events as well as elaboration on previously mentioned topics.

In the most recent article, we can see that Walz’s endorsement of President Obama’s candidacy in 2008, as well as President Trump appointing Walz to “the bipartisan Council of Governors”, are relatively recent additions. These additions are shown in Figure 12. Neither former president was mentioned in the oldest article scraped. Another section has also been added regarding Walz’s actions

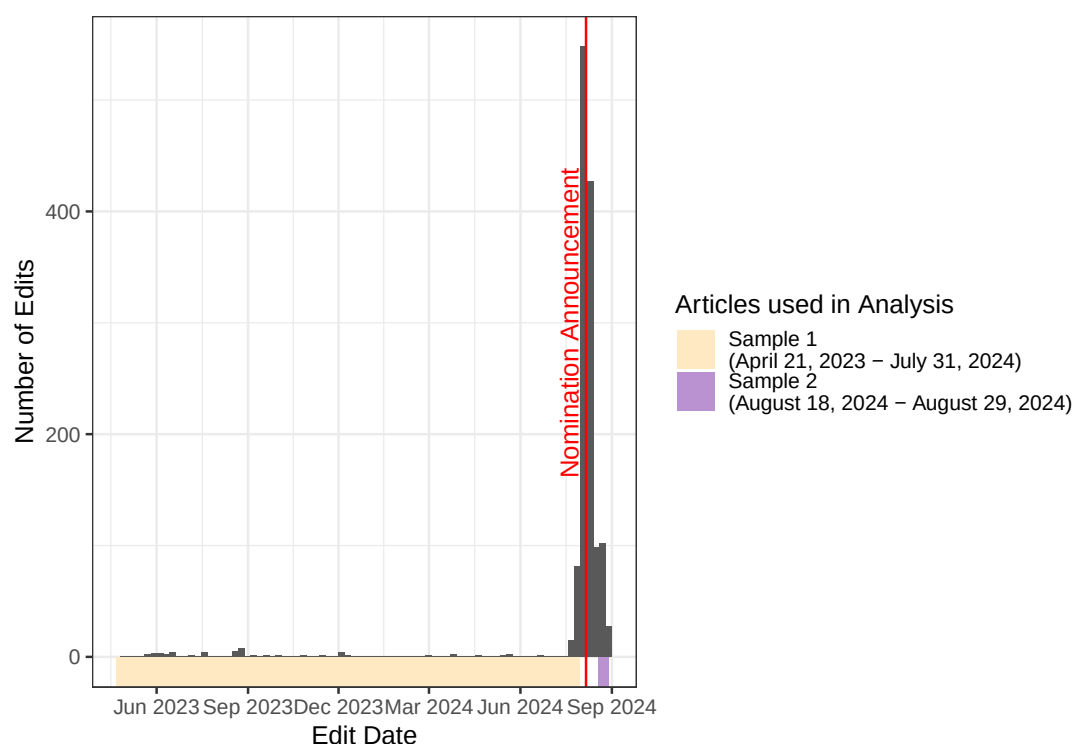


Figure 10: Number of Edits between April 22, 2023 and August 29, 2024. Sample 1, corresponding to articles before nomination, ranges from April 21, 2023 to July 31, 2024. Sample 2, corresponding to articles after nomination, ranges from August 18, 2024 to August 29, 2024. The red line indicates the date Walz’s nomination was announced (August 6, 2024).

related to tribal nations. These items were not referenced in the oldest article scraped (from April 22, 2023), despite most of these actions predating the article. This paragraph is shown in Figure 13.

The most notable phrase that is not commonly repeated from the oldest article in the database is that Walz “has been endorsed by the NRA multiple times.” In comparing with the August 29, 2024 article, we can see that more detail has been added, stating that Walz later denounced the NRA and believes in gun regulation. This comparison can be seen in Figure 14.

5 Conclusion

The package **highlightr** has diverse applications that can help researchers synthesize text data to evaluate changes, assess similarities, and highlight common information across documents. This package works by evaluating the frequency with which collocations from a source document appear in derivative documents, mapping this frequency to a color scale, and creating a gradient output of these frequencies to form a highlighting effect in HTML outputs. These applications have been demonstrated on the author’s submitted abstracts, as well as the Wikipedia article for Tim Walz.

While originally developed to compare participant notes to an online transcript, this package can generally be used to compare texts. In an extension of the abstract comparison, this method can be used to compare revised articles to earlier iterations, demonstrating which portions are consistent across edits as well as which portions have been updated. This visualization method has applications in visualizing borrowed phrases in literature and newspapers, as described by [Smith et al. \(2013\)](#) and [Olsen et al. \(2011\)](#). Similarly, this method can be applied to differing versions of the same text, such as [Fry \(2018\)](#)’s analysis of *On the Origin of Species*, to visualize the passages from the first version that were retained, or [Jänicke et al. \(2014\)](#)’s analysis of Bible passages. This form of literature analysis can also extend to versions of fairy tales in order to visualize changes over time. With some modifications to the string division method, this visualization method would also have applications in bioinformatics. This method provides a new way to visualize an entire document with regards to several derivative documents by mapping average phrase frequency directly onto the source document through a color gradient. Popular and unpopular phrases are instantly identifiable through color indicators, while the overall document view provides context.



Figure 11: The highlighted versions of the most recent (left) and oldest (right) articles about Tim Walz used in this analysis.

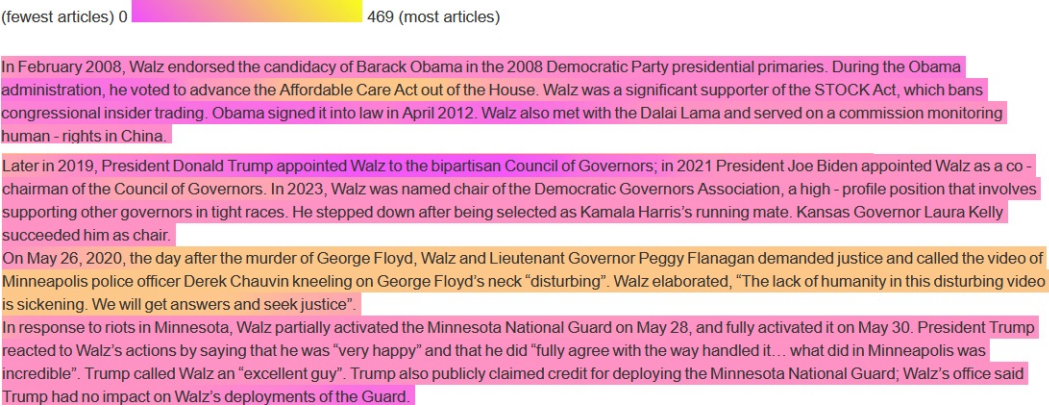


Figure 12: Newer additions to Walz’s Wikipedia page relating to former presidents. The purple highlighting indicates less frequent mentions

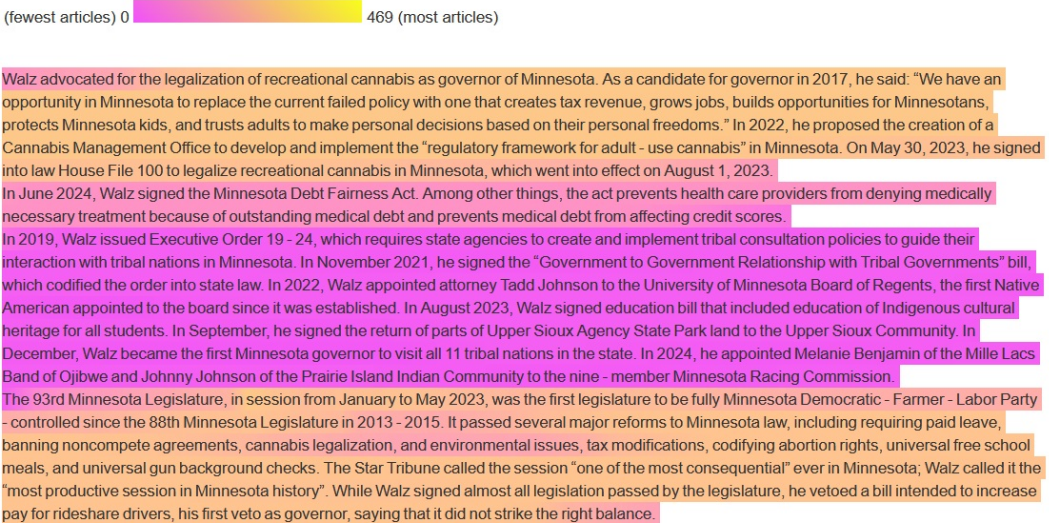


Figure 13: Highlighted comparison with the August 19, 2024 Wikipedia page as the source document. The purple indicates that this paragraph appeared in fewer previous articles than the surrounding paragraphs.



Figure 14: A Comparison between how Walz’s NRA support was described on Wikipedia in April 2023 versus August 2024. Purple indicates a lower frequency, while yellow indicates a higher frequency.

6 Acknowledgements

This work was funded (or partially funded) by the Center for Statistics and Applications in Forensic Evidence (CSAFE) through Cooperative Agreements 70NANB15H176 and 70NANB20H019 between NIST and Iowa State University, which includes activities carried out at Carnegie Mellon University, Duke University, University of California Irvine, University of Virginia, West Virginia University, University of Pennsylvania, Swarthmore College and University of Nebraska, Lincoln.

This paper and corresponding examples used R (R Core Team, 2025) and the following packages: **DiagrammeR** (Iannone and Roy, 2024), **gt** (Iannone et al., 2026), **highlightr** (Center for Statistics and Applications in Forensic Evidence et al., 2024), **imager** (Barthelme, 2025), **knitr** (Xie, 2025), **pander** (Daróczy and Tsegelskyi, 2025), **patchwork** (Pedersen, 2025), **plotly** (Sievert, 2020), **purrr** (Wickham and Henry, 2025), **rjtools** (O'Hara-Wild et al., 2025), **rmarkdown** (Allaire et al., 2025), **rvest** (Wickham, 2025a), **stringr** (Wickham, 2025b), **tidyverse** (Wickham et al., 2019), and **webshot2** (Chang, 2025).

References

- Tim Walz, Aug. 2024. URL https://en.wikipedia.org/wiki/Tim_Walz. [p64]
- J. Allaire, Y. Xie, C. Dervieux, J. McPherson, J. Luraschi, K. Ushey, A. Atkins, H. Wickham, J. Cheng, W. Chang, and R. Iannone. *rmarkdown: Dynamic Documents for R*, 2025. URL <https://github.com/rstudio/rmarkdown>. R package version 2.30. [p68]
- A. Amir, P. Charalampopoulos, S. P. Pissis, and J. Radoszewski. Longest Common Substring Made Fully Dynamic, July 2018. URL <http://arxiv.org/abs/1804.08731>. arXiv:1804.08731 [cs]. [p56]
- S. Barthelme. *imager: Image Processing Library Based on 'CImg'*, 2025. URL <https://CRAN.R-project.org/package=imager>. R package version 1.0.8. [p68]
- K. Benoit, K. Watanabe, H. Wang, P. Nulty, A. Obeng, S. Müller, and A. Matsuo. quanteda: An R package for the quantitative analysis of textual data. *Journal of Open Source Software*, 3(30):774, 2018. doi: 10.21105/joss.00774. URL <https://quanteda.io>. [p57]
- Center for Statistics and Applications in Forensic Evidence, R. Rogers, and S. VanderPlas. *highlightr: Highlight Conserved Edits Across Versions of a Document*, 2024. URL <https://CRAN.R-project.org/package=highlightr>. R package version 1.0.2. [p55, 68]
- W. Chang. *webshot2: Take Screenshots of Web Pages*, 2025. URL <https://CRAN.R-project.org/package=webshot2>. R package version 0.1.2. [p68]
- M. Crochemore, C. S. Iliopoulos, A. Langiu, and F. Mignosi. The longest common substring problem. *Mathematical Structures in Computer Science*, 27(2):277–295, Feb. 2017. ISSN 0960-1295, 1469-8072. doi: 10.1017/S0960129515000110. URL <http://www.cambridge.org/core/journals/mathematical-structures-in-computer-science/article/longest-common-substring-problem/1DA95504B11A551E2BD130AA834CF41>. Publisher: Cambridge University Press. [p56]
- G. Daróczy and R. Tsegelskyi. *pander: An R 'Pandoc' Writer*, 2025. URL <https://CRAN.R-project.org/package=pander>. R package version 0.6.6. [p68]
- J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In J. Burstein, C. Doran, and T. Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423/>. [p58]
- I. Fellows. *wordcloud: Word Clouds*, 2018. URL <https://CRAN.R-project.org/package=wordcloud>. R package version 2.6. [p55]
- B. Fry. Tracing the Origin, May 2018. URL https://medium.com/@ben_fry/tracing-the-origin-65011dc20877. [p56, 65]
- A. Gelman, C. Pasarica, and R. Dodhia. Let's Practice What We Preach: Turning Tables into Graphs. *The American Statistician*, 56(2):121–130, 2002. ISSN 0003-1305. URL <https://www.jstor.org/stable/3087382>. Publisher: [American Statistical Association, Taylor & Francis, Ltd.]. [p57]

- M. Gleicher, D. Albers, R. Walker, I. Jusufi, C. D. Hansen, and J. C. Roberts. Visual comparison for information visualization. *Information Visualization*, 10(4):289–309, Oct. 2011. ISSN 1473-8716, 1473-8724. doi: 10.1177/1473871611416549. URL <https://journals.sagepub.com/doi/10.1177/1473871611416549>. [p56]
- B. Green. *zoomerjoin: Superlatively Fast Fuzzy Joins*, 2025. URL <https://CRAN.R-project.org/package=zoomerjoin>. R package version 0.2.1. [p56, 57, 58]
- R. Iannone and O. Roy. *DiagrammeR: Graph/Network Visualization*, 2024. URL <https://CRAN.R-project.org/package=DiagrammeR>. R package version 1.0.11. [p68]
- R. Iannone, J. Cheng, B. Schloerke, S. Haughton, E. Hughes, A. Lauer, R. François, J. Seo, K. Brevoort, and O. Roy. *gt: Easily Create Presentation-Ready Display Tables*, 2026. URL <https://CRAN.R-project.org/package=gt>. R package version 1.3.0. [p68]
- S. Jänicke, A. Geßner, M. Büchler, and G. Scheuermann. Visualizations for Text Re-use. In *2014 International Conference on Information Visualization Theory and Applications (IVAPP)*, pages 59–70, Jan. 2014. URL <https://ieeexplore.ieee.org/document/7294398/>. [p56, 65]
- S. Lin, J. Fortuna, C. Kulkarni, M. Stone, and J. Heer. Selecting Semantically-Resonant Colors for Data Visualization. *Computer Graphics Forum*, 32(3pt4):401–410, 2013. ISSN 1467-8659. doi: 10.1111/cgf.12127. URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.12127>. _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.12127>. [p57]
- Merriam-Webster. Definition of COLLOCATION, Apr. 2023. URL <https://www.merriam-webster.com/dictionary/collocation>. [p56]
- S. R. Midway. Principles of Effective Data Visualization. *Patterns*, 1(9):100141, Dec. 2020. ISSN 2666-3899. doi: 10.1016/j.patter.2020.100141. URL <https://www.sciencedirect.com/science/article/pii/S2666389920301896>. [p57]
- M. O'Hara-Wild, S. Kobakian, H. S. Zhang, D. Cook, S. Urbanek, and C. Dervieux. *rjtools: Preparing, Checking, and Submitting Articles to the 'R Journal'*, 2025. URL <https://CRAN.R-project.org/package=rjtools>. R package version 1.0.18.1. [p68]
- M. Olsen, R. Horton, and G. Roe. Something Borrowed: Sequence Alignment and the Identification of Similar Passages in Large Text Collections. *Digital Studies / Le champ numérique*, 2(1), May 2011. ISSN 1918-3666. doi: 10.16995/dscn.258. URL <https://www.digitalstudies.org/article/id/7224/>. [p56, 65]
- A. Parker and J. Hamblen. Computer algorithms for plagiarism detection. *IEEE Transactions on Education*, 32(2):94–99, May 1989. ISSN 1557-9638. doi: 10.1109/13.28038. Conference Name: IEEE Transactions on Education. [p56]
- T. L. Pedersen. *patchwork: The Composer of Plots*, 2025. URL <https://CRAN.R-project.org/package=patchwork>. R package version 1.3.2. [p68]
- S. S. L. Piao and T. McEnery. A Tool for Text Comparison. *Proceedings of the Corpus Linguistics*, 2003. [p56]
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2025. URL <https://www.R-project.org/>. [p68]
- R. Rogers. Measuring Jury Perception of Explainable Machine Learning and Demonstrative Evidence. *Dissertations and Doctoral Documents from University of Nebraska-Lincoln*, 2023–, Apr. 2024. URL <https://digitalcommons.unl.edu/dissunl/69>. [p59]
- E. Schubert. Stop using the elbow criterion for k-means and how to choose the number of clusters instead. *SIGKDD Expl. Newsl.*, 25(1):36–42, July 2023. ISSN 1931-0145. doi: 10.1145/3606274.3606278. URL <https://dl.acm.org/doi/10.1145/3606274.3606278>. [p59]
- M. Schweinberger. *Analyzing Co-Occurrences and Collocations in R*. The University of Queensland, Australia. School of Languages and Cultures, Brisbane, 2023.05.31 edition, 2023. <https://ladal.edu.au/coll.html>. [p57]
- Y. Shibuya and K. E. Jensen. Mining for constructions in texts using N-gram and network analysis. *Globe: A Journal of Language, Culture and Communication*, 2, Oct. 2015. ISSN 2246-8838. doi: 10.5278/ojs.globe.v2i0.1113. URL <https://journals.aau.dk/index.php/globe/article/view/1113>. [p55, 56]

- C. Sievert. *Interactive Web-Based Data Visualization with R, plotly, and shiny*. Chapman and Hall/CRC, 2020. ISBN 9781138331457. URL <https://plotly-r.com>. [p68]
- J. Silge. She Giggles, He Gallops. URL <https://pudding.cool/2017/08/screen-direction>. [p55]
- D. A. Smith, R. Cordell, and E. M. Dillon. Infectious texts: Modeling text reuse in nineteenth-century newspapers. In *2013 IEEE International Conference on Big Data*, pages 86–94, Oct. 2013. doi: 10.1109/BigData.2013.6691675. URL <https://ieeexplore.ieee.org/document/6691675/>. [p56, 58, 65]
- C. Thomas and B. Sheldon. The "Knee of a Curve"—Useful Clue but Incomplete Support. *Military Operations Research*, 4(2):17–24, 1999. ISSN 1082-5983. URL <https://www.jstor.org/stable/43940795>. Publisher: Military Operations Research Society. [p59]
- H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. ISBN 978-3-319-24277-4. URL <https://ggplot2.tidyverse.org>. [p55]
- H. Wickham. *rvest: Easily Harvest (Scrape) Web Pages*, 2025a. URL <https://CRAN.R-project.org/package=rvest>. R package version 1.0.5. [p68]
- H. Wickham. *stringr: Simple, Consistent Wrappers for Common String Operations*, 2025b. URL <https://CRAN.R-project.org/package=stringr>. R package version 1.6.0. [p68]
- H. Wickham and L. Henry. *purrr: Functional Programming Tools*, 2025. URL <https://CRAN.R-project.org/package=purrr>. R package version 1.0.4. [p68]
- H. Wickham, M. Averick, J. Bryan, W. Chang, L. D. McGowan, R. François, G. Golemund, A. Hayes, L. Henry, J. Hester, M. Kuhn, T. L. Pedersen, E. Miller, S. M. Bache, K. Müller, J. Ooms, D. Robinson, D. P. Seidel, V. Spinu, K. Takahashi, D. Vaughan, C. Wilke, K. Woo, and H. Yutani. Welcome to the tidyverse. *Journal of Open Source Software*, 4(43):1686, 2019. doi: 10.21105/joss.01686. [p68]
- J. Wijnffels and K. Watanabe. *word2vec: Distributed Representations of Words*, 2025. URL <https://CRAN.R-project.org/package=word2vec>. R package version 0.4.1. [p58]
- W. Wu, B. Li, L. Chen, J. Gao, and C. Zhang. A Review for Weighted MinHash Algorithms. *IEEE Transactions on Knowledge and Data Engineering*, 34(6):2553–2573, June 2022. ISSN 1558-2191. doi: 10.1109/TKDE.2020.3021067. URL <https://ieeexplore.ieee.org/abstract/document/9184977>. [p58]
- Y. Xie. *knitr: A General-Purpose Package for Dynamic Report Generation in R*, 2025. URL <https://yihui.org/knitr/>. R package version 1.51. [p68]

Rachel Rogers
University of Technology-Sydney
School of Mathematical and Physical Sciences
Sydney, New South Wales, Australia
ORCID: 0000-0002-4145-9630
rachel.rogers@uts.edu.au

Susan VanderPlas
University of Nebraska-Lincoln
Department of Statistics
Lincoln, Nebraska, United States of America
ORCID: 0000-0002-3803-0972
susan.vanderplas@unl.edu